In [1]:
```
from pdf2image import convert_from_path
from IPython.display import display
images = convert_from_path(r"C:\Users\miles\BYU-Idaho\11th Semester\336 Advanced Lab\W0
for img in images:
    display(img)
```

Physics 336
HW9

Solve the following problems. Do at least one in Mathematica, and at least one in Python.

1. Recall that in a series RLC circuit, the impedance is expected to follow

$$Z = \sqrt{R^2 + \left(2\pi f L - \frac{1}{2\pi f C}\right)^2}$$

The resistor voltage, $V_R = IR = (V_0/Z)R$ can be measured over a range of frequencies to determine the values of L, and C. Suppose the frequency is adjusted in 50-Hz increments, and V_R is measured:

$f(\text{Hz}) = \{50, 100, 150, 200, 250, 300, 350, 400, 450, 500, 550, 600, 650, 700, 750\}$
$V_R(\text{V}) = \{0.40, 0.83, 1.20, 1.49, 1.67, 1.74, 1.74, 1.68, 1.60, 1.51, 1.42, 1.34, 1.26, 1.18, 1.11\}$

Suppose we measure $V_0 = 1.80$ V and $R = 17.0\ \Omega$ using a multimeter. Analyze these data to determine values for L and C.

2. The activity of a mixture of *two* radioisotopes is measured using a Geiger counter, with counts measured in forty 10-second intervals, centered one-minute apart (beginning at 1.00 min)

$$\begin{aligned} R(\text{counts}) = \{&2343, 2066, 1839, 1690, 1490, 1397, 1232, 1171, 1017, 1000, 956, 864,\\ &795, 793, 748, 736, 703, 662, 636, 622, 564, 567, 541, 512, 534, 497,\\ &504, 456, 466, 469, 434, 410, 421, 400, 413, 380, 348, 376, 341, 344\} \end{aligned}$$

Fit this data to an appropriate function, and interpret the results.

3. A portion of the solar spectrum near an absorption line has been measured.

$$\begin{aligned} \lambda(\text{nm}) = \{&740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755,\\ &756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771,\\ &772, 773, 774, 775, 776, 777, 778, 779, 780\} \pm 1 \end{aligned}$$

$$\begin{aligned} I(\text{arbitrary units}) = \{&0.277, 0.275, 0.273, 0.271, 0.269, 0.266, 0.263, 0.259, 0.256, 0.252, 0.248, 0.244, 0.240,\\ &0.237, 0.232, 0.222, 0.202, 0.179, 0.159, 0.138, 0.120, 0.105, 0.099, 0.107, 0.119, 0.131,\\ &0.143, 0.154, 0.161, 0.166, 0.166, 0.165, 0.163, 0.160, 0.158, 0.155, 0.152, 0.149, 0.146,\\ &0.144, 0.141\} \pm 0.003 \end{aligned}$$

Treat this as a Gaussian *absorption* with a background, and analyze this data.

4. A distant star is a binary system. It is imaged with a telescope, and the counts per pixel are measured along a line through the center of the image.

   $\text{pixel}, p = \{22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43\}$
   $\text{counts}, c = \{1, 3, 7, 14, 26, 43, 64, 84, 99, 106, 102, 92, 82, 77, 75, 70, 56, 37, 19, 8, 3, 1\}$

   Assuming the light from each star creates a different Gaussian distribution (with unknown amplitude A, mean μ, and standard deviation σ) across pixels, fit this data to find the relative brightness of these two stars.

5. A converging lens is used to cast real images on a screen. The object and image distances are recorded.

$$\begin{aligned} d_o(\text{cm}) =& \{18.0, 21.0, 24.0, 27.0, 30.0, 40.0, 46.0, 50.0, 53.0\} \ \pm 0.1 \,\text{each} \\ d_i(\text{cm}) =& \{101 \pm 3, 56 \pm 2, 43.0 \pm 1.5, 35.5 \pm 0.7, 31.1 \pm 0.6, 25.2 \pm 0.4, \\ & 23.1 \pm 0.3, 22.2 \pm 0.3, 21.9 \pm 0.3\} \end{aligned}$$

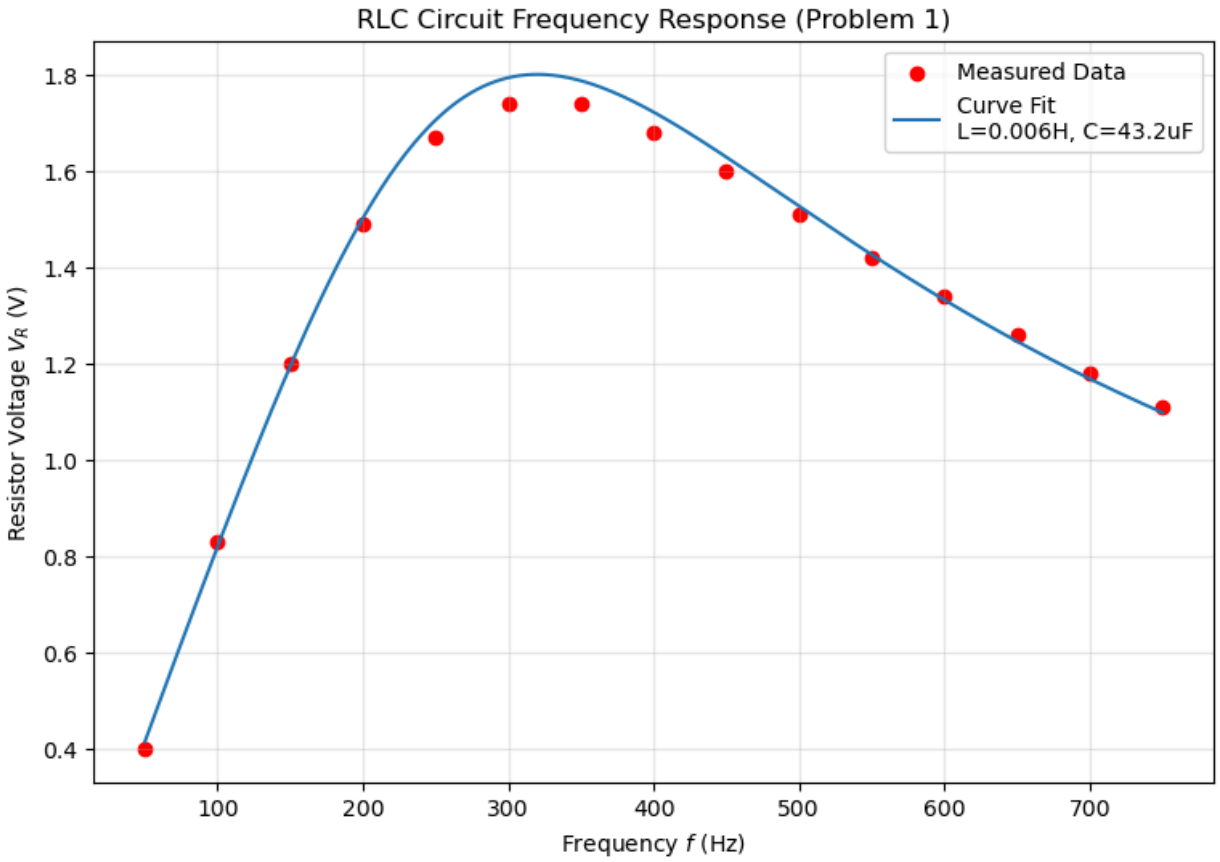
   Plot d_i *vs.* d_o, and fit it it to an appropriate function. Use this to determine the power (in diopters) of the lens.

Problem 1.

In [16]:
```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

f=np.array([50,100,150,200,250,300,350,400,450,500,550,600,650,700,750])
V=np.array([0.4,0.83,1.2,1.49,1.67,1.74,1.74,1.68,1.60,1.51,1.42,1.34,1.26,1.18,1.11])
```

```
V0=1.8
R=17.0

def Vr(f,L,C):
    Z=np.sqrt(R**2+(2*np.pi*f*L-1/(2*np.pi*f*C))**2)
    VR=V0*R/Z
    return VR

L_guesses=np.linspace(0.005,0.006,100)
C_guesses=np.linspace(43e-6,44e-6,100)

min_guess=float('inf')

for i in L_guesses:
    for j in C_guesses:
        Vr_predicted=Vr(f,i,j)
        guess=np.sum((V-Vr_predicted)**2)
        if guess < min_guess:
            min_guess=guess
            best_L,best_C= i,j

initial_guesses=[best_L,best_C]

popt, pcov = curve_fit(Vr, f, V, p0=initial_guesses)

L_fit, C_fit = popt
dL, dC = np.sqrt(np.diag(pcov)) # Standard deviations of the parameters

print(f"Fit Results:")
print(f"L = {L_fit:.4f} ± {dL:.4f} H")
print(f"C = {C_fit*1e6:.2f} ± {dC*1e6:.2f} uF")

# 4. Plotting the results
f_fine = np.linspace(50, 750, 500)
Vr_fit = Vr(f_fine, L_fit, C_fit)

plt.figure(figsize=(9, 6))
plt.scatter(f, V, color='red', label='Measured Data')
plt.plot(f_fine, Vr_fit, label=f'Curve Fit\nL={L_fit:.3f}H, C={C_fit*1e6:.1f}uF')
plt.xlabel('Frequency $f$ (Hz)')
plt.ylabel('Resistor Voltage $V_R$ (V)')
plt.title('RLC Circuit Frequency Response (Problem 1)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
Fit Results:
L = 0.0057 ± 0.0001 H
C = 43.15 ± 0.88 uF
```



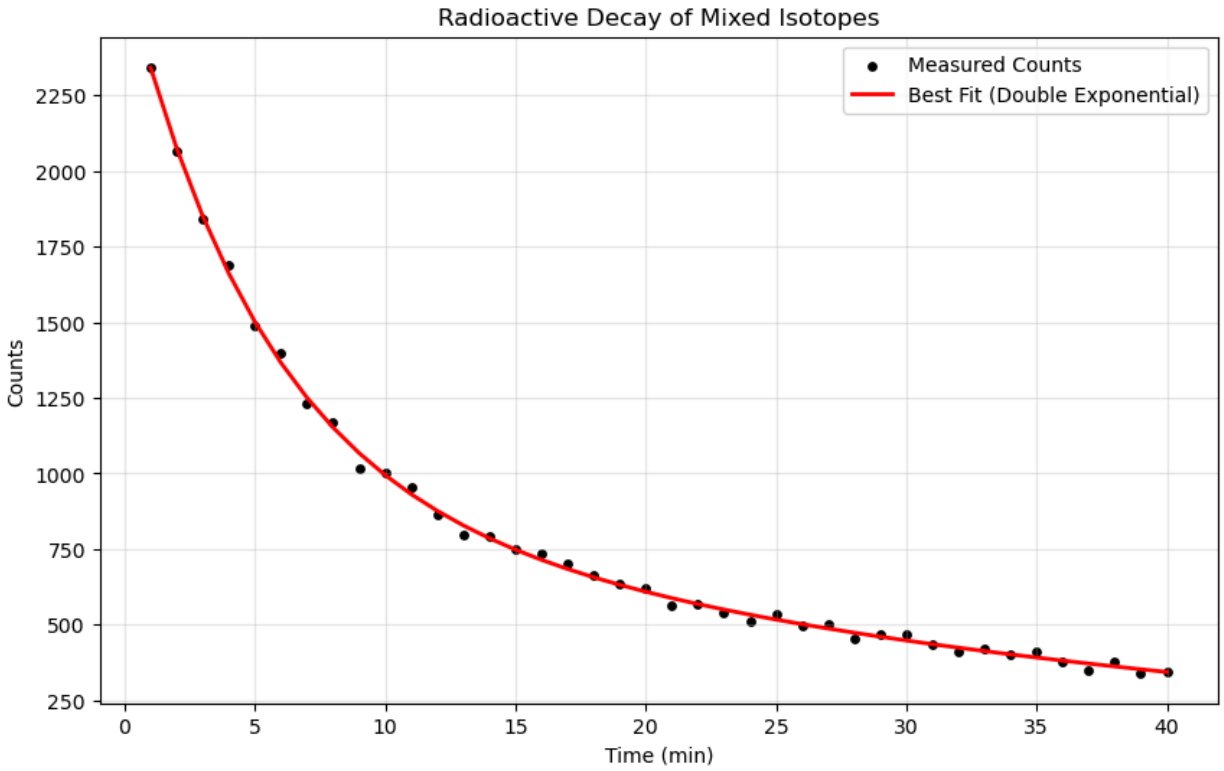
Problem 2.

In [24]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

R=np.array([2343, 2066, 1839, 1690, 1490, 1397, 1232, 1171, 1017, 1000, 956, 864,
            795, 793, 748, 736, 703, 662, 636, 622, 564, 567, 541, 512, 534, 497,
            504, 456, 466, 469, 434, 410, 421, 400, 413, 380, 348, 376, 341, 344])

t=np.arange(1.0,41.0,1.0)

def decay(t,A,L1,B,L2,C):
    return A*np.exp(-L1*t)+B*np.exp(-L2*t)+C

C_g=300
A_g=1500
B_g=800

L2_g=np.linspace(0.01,0.1,100)
min_guess_B=float('inf')

for i in L2_g:
    B_guess= (R[20]-C_g)/np.exp(-i*t[20])
    pred = B_guess* np.exp(-i * t[20:]) + C_g
    guess = np.sum((R[20:] - pred)**2)
    if guess < min_guess_B:
        min_guess_B = guess
        best_L2=i
```

```
R_A=R[:10]-(1000*np.exp(-best_L2*t[:10])+C_g)
L1_g=np.linspace(0.1,1,100)
min_guess_A=float('inf')

for i in L1_g:
    A_guess = R_A[0]/np.exp(-i * t[0])
    pred = A_guess * np.exp(-i * t[:10])
    guess = np.sum((R_A-pred)**2)
    if guess < min_guess_A:
        min_guess_A = guess
        best_L1=i

initial_p0 = [A_g, best_L1, B_g, best_L2, C_g]
# Set bounds: (lower_limits, upper_limits)
# Parameters are [A, L1, B, L2, C]
lower_bounds = [0, 0, 0, 0, 0]
upper_bounds = [5000, 2.0, 5000, 0.5, 400]

popt, pcov = curve_fit(decay, t, R, p0=initial_p0, bounds=(lower_bounds, upper_bounds))

A, L1, B, L2, C = popt
errors = np.sqrt(np.diag(pcov))

print(f"Final Parameters:")
print(f" A = {A:.1f}, L1 = {L1:.4f}")
print(f" B = {B:.1f}, L2 = {L2:.4f}")
print(f" C = {C:.1f}")

# Half-lives
thalf1 = np.log(2) / L1
thalf2 = np.log(2) / L2
print(f"\nHalf-life 1: {thalf1:.2f} min")
print(f"Half-life 2: {thalf2:.2f} min")

# Plotting
plt.figure(figsize=(10, 6))
plt.scatter(t, R, label='Measured Counts', color='black', s=15)
plt.plot(t, decay(t, *popt), label='Best Fit (Double Exponential)', color='red', linewi
plt.xlabel('Time (min)')
plt.ylabel('Counts')
plt.title('Radioactive Decay of Mixed Isotopes')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()
```

```
Final Parameters:
 A = 1713.3, L1 = 0.1882
 B = 945.6, L2 = 0.0253
 C = 0.0

Half-life 1: 3.68 min
Half-life 2: 27.34 min
```



Problem 3.

In [25]:
```
from pdf2image import convert_from_path
from IPython.display import display
images = convert_from_path(r"C:\Users\miles\BYU-Idaho\11th Semester\336 Advanced Lab\W08
for img in images:
    display(img)
```

HW09; Problem 3, Absorption Spectra

3. A portion of the solar spectrum near an absorption line has been measured.

$$\begin{aligned}\lambda(\text{nm}) =& \{740, 741, 742, 743, 744, 745, 746, 747, 748, 749, 750, 751, 752, 753, 754, 755,\\ & 756, 757, 758, 759, 760, 761, 762, 763, 764, 765, 766, 767, 768, 769, 770, 771,\\ & 772, 773, 774, 775, 776, 777, 778, 779, 780\} \pm 1\\ I(\text{arbitrary units}) =& \{0.277, 0.275, 0.273, 0.271, 0.269, 0.266, 0.263, 0.259, 0.256, 0.252, 0.248, 0.244, 0.240,\\ & 0.237, 0.232, 0.222, 0.202, 0.179, 0.159, 0.138, 0.120, 0.105, 0.099, 0.107, 0.119, 0.131,\\ & 0.143, 0.154, 0.161, 0.166, 0.166, 0.165, 0.163, 0.160, 0.158, 0.155, 0.152, 0.149, 0.146,\\ & 0.144, 0.141\} \pm 0.003\end{aligned}$$

Treat this as a Gaussian *absorption* with a background, and analyze this data.

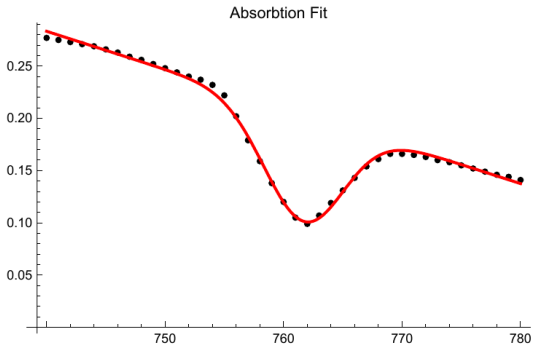
In[55]:=

```
lambda = Range[740, 780, 1];
i = {0.277, 0.275, 0.273, 0.271, 0.269, 0.266, 0.263, 0.259, 0.256,
   0.252, 0.248, 0.244, 0.240, 0.237, 0.232, 0.222, 0.202, 0.179, 0.159, 0.138,
   0.120, 0.105, 0.099, 0.107, 0.119, 0.131, 0.143, 0.154, 0.161, 0.166, 0.166,
   0.165, 0.163, 0.160, 0.158, 0.155, 0.152, 0.149, 0.146, 0.144, 0.141};
data = Transpose[{lambda, i}];
(*Define the model:Linear Background-Gaussian Dip*)
model = m * x + b - h * Exp[- (x - mu)^2 / (2 * sigma^2)];

(*Perform the fit with initial guesses*)
nlm = NonlinearModelFit[data, model,
   {{m, -0.003}, {b, 2.5}, {h, 0.17}, {mu, 762}, {sigma, 3}}, x];

(*Display the results*)
nlm["ParameterTable"]
Show[ListPlot[data, PlotStyle → Black, PlotLabel → "Absorbtion Fit"],
 Plot[nlm[x], {x, 740, 780},  PlotStyle → Red]]
```

Out[60]=

	Estimate	Standard Error	t-Statistic	P-Value
m	−0.00364471	0.0000424498	−85.8594	3.02537×10^{-43}
b	2.98042	0.032214	92.5192	2.0793×10^{-44}
h	0.102819	0.0016352	62.8789	2.08469×10^{-38}
mu	761.671	0.0602916	12 633.1	3.01869×10^{-121}
sigma	3.34374	0.0661975	50.5116	5.07578×10^{-35}

Out[61]=



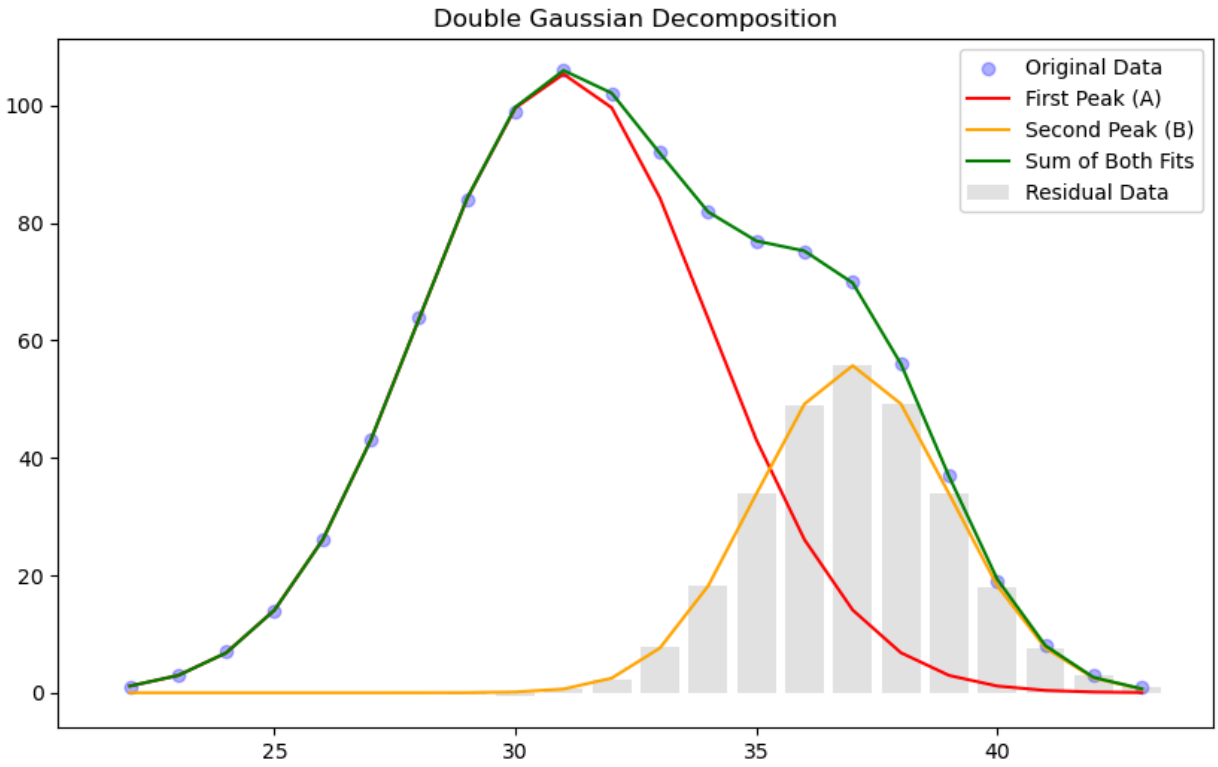
Problem 4.

In [63]:

```
import numpy as np
from matplotlib import pyplot as plt
from scipy.optimize import curve_fit
from numpy.random import normal

pixel=np.arange(22,44)
```

```
counts=np.array([1, 3, 7, 14, 26, 43, 64, 84, 99, 106, 102,
        92, 82, 77, 75, 70, 56, 37, 19, 8, 3, 1])


max_index=np.argmax(counts)
max_c=max(counts)
peak_pixel=pixel[max_index]
pA=pixel[:max_index+1]
cA=counts[:max_index+1]

def gauss(x,A,x1,sigma):
    return A*np.exp(-(x-x1)**2/2/sigma**2)

initial_guess=[max_c,peak_pixel,2]
lower_bounds = [0, peak_pixel - 0.001, 0.1]
upper_bounds = [np.inf, peak_pixel + 0.001, np.inf]
popt,pcov = curve_fit(gauss, pA,cA, p0=initial_guess, bounds=(lower_bounds, upper_bounds

B_counts=counts-gauss(pixel, *popt)
initial_guess_B = [20,35,3]
popt2,pcov2 = curve_fit(gauss, pixel, B_counts, p0=initial_guess_B)

combined=B_counts+gauss(pixel, *popt)

print(f"Final Parameters:")
print(f"Peak 1 Amplitude:{popt[0]:.2f} Mean: {popt[1]:.2f}, Std:{popt[2]:.2f} ")
print(f"Peak 2 Amplitude:{popt2[0]:.2f} Mean: {popt2[1]:.2f}, Std:{popt2[2]:.2f} ")



plt.figure(figsize=(10, 6))
plt.scatter(pixel, counts, label='Original Data', color="blue", alpha=0.3)
plt.plot(pixel, gauss(pixel, *popt), color='red', label='First Peak (A)')
plt.plot(pixel, gauss(pixel, *popt2), color='orange', label='Second Peak (B)')
plt.plot(pixel, gauss(pixel, *popt) + gauss(pixel, *popt2), color='green', label='Sum of


plt.legend()
plt.title("Double Gaussian Decomposition")
plt.show()
```

```
Final Parameters:
Peak 1 Amplitude:105.37 Mean: 31.00, Std:2.99
Peak 2 Amplitude:55.72 Mean: 37.00, Std:2.01
```



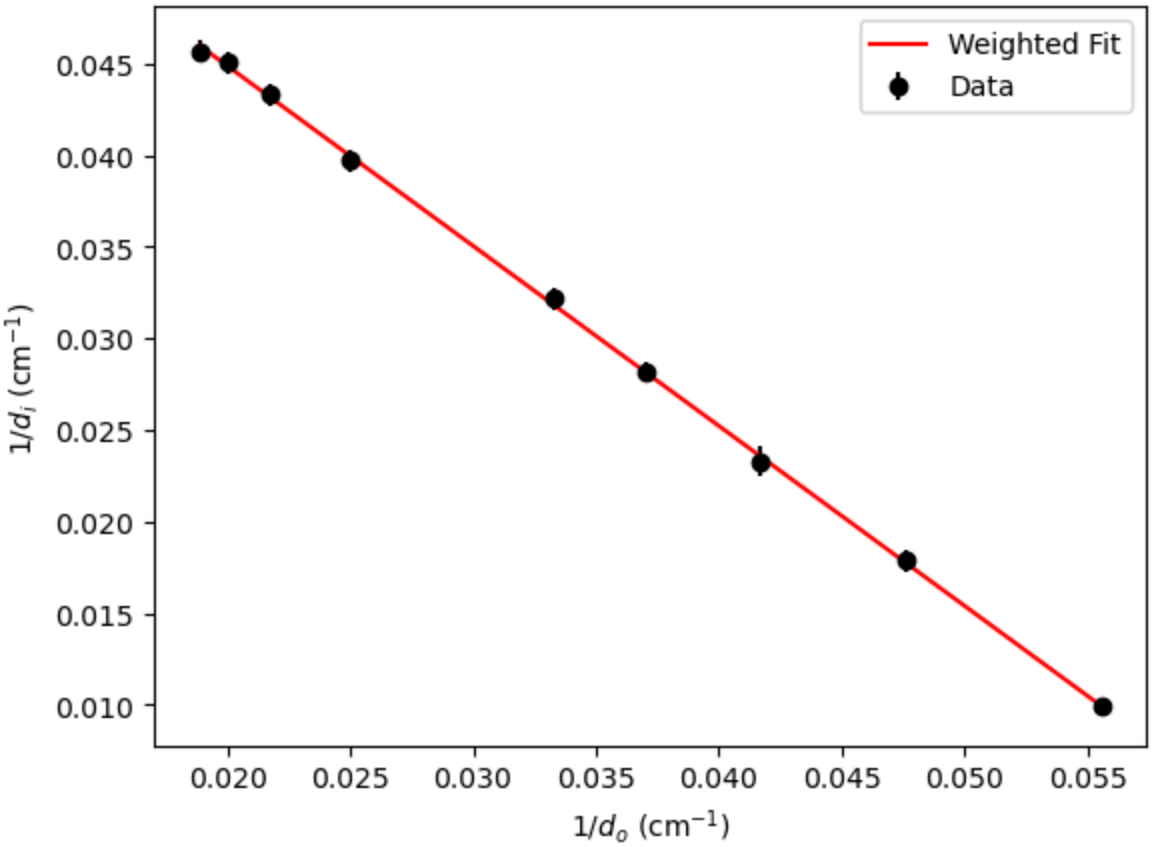
Problem 5.

In [65]:

```
import numpy as np
import matplotlib.pyplot as plt

# Data
do = np.array([18.0, 21.0, 24.0, 27.0, 30.0, 40.0, 46.0, 50.0, 53.0])
ddo = 0.1 # cm

di = np.array([101.0, 56.0, 43.0, 35.5, 31.1, 25.2, 23.1, 22.2, 21.9])
ddi = np.array([3.0, 2.0, 1.5, 0.7, 0.6, 0.4, 0.3, 0.3, 0.3])

# Transform to linear space (Units: 1/cm)
x = 1/do
y = 1/di

# Note: Since do errors are very small,
# we focus on di errors for the weights
sigma_y = ddi / (di**2)
weights = 1 / sigma_y

# Weighted Polyfit (degree 1 for a line)
# cov=True allows us to get the uncertainty of the fit parameters
params, cov = np.polyfit(x, y, 1, w=weights, cov=True)

m, b = params
dm, db = np.sqrt(np.diag(cov))

# Power is the intercept 'b' in units of 1/cm.
# Convert to Diopters (1/m) by multiplying by 100.
power_diopters = b * 100
power_error = db * 100
```

```
print(f"Slope (should be ~ -1): {m:.3f} ± {dm:.3f}")
print(f"Lens Power: {power_diopters:.2f} ± {power_error:.2f} Diopters")

# Plotting
plt.errorbar(x, y, yerr=sigma_y, fmt='ko', label='Data')
plt.plot(x, m*x + b, 'r-', label='Weighted Fit')
plt.xlabel('$1/d_o$ (cm$^{-1}$)')
plt.ylabel('$1/d_i$ (cm$^{-1}$)')
plt.legend()
plt.show()
```

```
Slope (should be ~ -1): -0.983 ± 0.005
Lens Power: 6.45 ± 0.02 Diopters
```



In [ ]: